

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA010 AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

*I **K Sathwik Kumar (192425051)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Sathwik Kumar

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

Student Academic Advisor Expert System Using Prolog

1. Introduction

- An expert system is an Artificial Intelligence system that uses a knowledge base and inference mechanisms to solve problems that normally require human expertise.
- This project develops a Student Academic Advisor Expert System using Prolog. The system analyzes student academic information such as attendance, internal marks, assignment performance, and learning difficulty.
- Based on these facts, the system applies production rules to derive conclusions such as academic risk, attendance improvement, extra learning support, assignment support, and faculty counselling.

The system demonstrates:

- Forward chaining
- Backward chaining
- Facts and rules
- Variables
- Unification
- Backtracking
- Logical inference
- Automated recommendation generation

2. Problem Statement

- Students may experience academic difficulties because of low attendance, poor internal marks, poor assignment performance, or learning difficulties.
- Manually identifying students who require academic support can be time-consuming.
- The objective of this project is to develop an intelligent academic advisory system that analyzes student information and automatically provides suitable academic recommendations.
- The system accepts student information as facts and uses predefined production rules to generate conclusions and recommendations.

Inputs

- Student name
- Attendance level
- Internal marks
- Assignment performance
- Learning difficulty

Outputs

- Academic Risk
- Improve Attendance
- Extra Learning Support
- Assignment Support
- Faculty Counselling
- Student Requires Academic Attention
- Student Performing Well

3. Objectives

1. To develop a rule-based expert system using Prolog.
2. To represent student academic information using facts.
3. To represent academic decision-making knowledge using production rules.
4. To implement forward chaining for data-driven reasoning.
5. To implement backward chaining for goal-driven reasoning.
6. To demonstrate unification and backtracking in Prolog.
7. To generate meaningful academic recommendations.

4. Domain Analysis

The selected domain is Student Academic Advising.

The expert system considers the following academic indicators:

Attribute	Possible Values	Meaning
Attendance	high / low	Indicates student's attendance level
Internal Marks	high / low	Indicates performance in internal assessments
Assignment Performance	good / poor	Indicates assignment quality
Learning Difficulty	yes / no	Indicates whether additional learning support may be required

The system combines these attributes through logical rules.

For example:

Low attendance + Low internal marks → Academic Risk

5. Knowledge Acquisition

- The knowledge represented in the system consists of academic advising conditions and their corresponding recommendations.

A student's academic information can be represented as:

student(rohan, low, low, poor, yes).

This represents:

- Student: Rohan
- Attendance: Low
- Internal Marks: Low
- Assignment Performance: Poor
- Learning Difficulty: Yes

```
39 ?- working_directory(_, 'D:/').
true.

40 ?- [implementation].
true.

41 ?- list_students.

=====
          STUDENTS IN KNOWLEDGE BASE
=====
- aditya
- ananya
- vikram
- kavya
- rohan
=====
true.
```

The expert knowledge is represented using production rules.

6. Knowledge Representation

6.1 Student Facts

student(Name, Attendance, Marks, Assignment, Difficulty).

Example:

student(rohan, low, low, poor, yes).

Other students can also be represented in the knowledge base to test different academic situations.

6.2 Production Rules

Academic Risk

academic_risk(Name) :-

student(Name, low, low, _, _).

Academic Risk Based on Attendance and Assignment

academic_risk(Name) :-

student(Name, low, _, poor, _).

Attendance Improvement

attendance_improvement(Name) :-

student(Name, low, _, _, _).

Extra Learning Support

extra_learning_support(Name) :-

student(Name, _, _, _, yes).

Assignment Support

assignment_support(Name) :-

student(Name, _, _, poor, _).

Faculty Counselling

faculty_counselling(Name) :-

academic_risk(Name).

Overall Good Performance

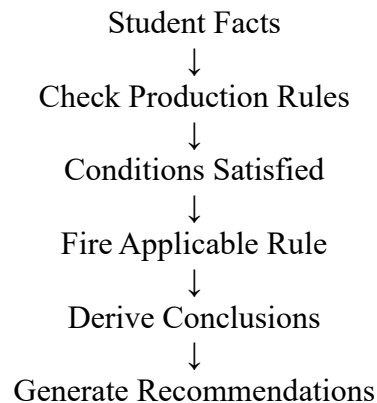
overall_good_performance(Name) :-

student(Name, high, high, good, no).

7. Forward Chaining

- Forward chaining is a data-driven reasoning approach.
- The system starts with known student facts and checks which rules can be applied.

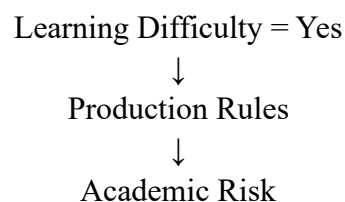
Process:



For Rohan:

- Low Attendance
- Low Internal Marks

Poor Assignment Performance

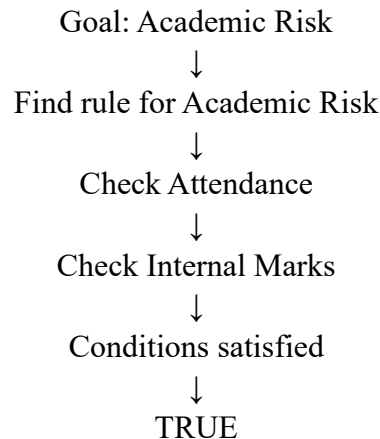


- Attendance Improvement
- Extra Learning Support
- Assignment Support
- Faculty Counselling

8. Backward Chaining

- Backward chaining is a goal-driven reasoning approach.
- The system begins with a goal and checks whether that goal can be proved from available facts and rules.

Reasoning:



Other goals include:

attendance_improvement(rohan).
extra_learning_support(rohan).
assignment_support(rohan).
faculty_counselling(rohan).
overall_good_performance(rohan).

9. Unification and Backtracking

- Unification
- Unification allows Prolog variables to match values.

Example:

- `?- academic_risk(X).`
- Here, **X is a variable**. Prolog searches for students satisfying the academic-risk rules.
- Example results can be:
- `X = aditya ;`
- `X = vikram ;`
- `X = rohan ;`
- `false.`

Backtracking

- When Prolog finds one solution, entering:
- `;`
- causes Prolog to search for another possible solution.

This demonstrates **variable matching, unification, multiple solutions, and backtracking** in Prolog.

This demonstrates:

- Variable matching
- Unification
- Multiple solutions
- Backtracking

```
42 ?- advisor(rohan).
```

```
*****
```

```
STUDENT ACADEMIC ADVISOR EXPERT SYSTEM
```

```
*****
```

```
=====
```

```
STUDENT INFORMATION
```

```
=====
```

```
Student Name      : rohan  
Attendance        : low  
Internal Marks    : low  
Assignment Performance : poor  
Learning Difficulty : yes
```

```
=====
```

10. Procedural and Non-Procedural Paradigm Analysis

- Procedural Approach
- A procedural approach describes how the problem should be solved.

Example:

- Read student information.
- Check attendance.
- Check internal marks.
- Check assignments.
- Check learning difficulty.
- Generate recommendations.
- The programmer controls the execution order.
- Non-Procedural / Declarative Approach
- Prolog follows a declarative programming style.

Example:

academic_risk(Name) :-

student(Name, low, low, _, _).

- The rule describes the relationship between student information and the conclusion.
- Prolog determines how to search for solutions using its inference mechanism.

Comparison

```
=====
                        FORWARD CHAINING
=====
Initial Facts:
Attendance           = low
Internal Marks       = low
Assignment Performance = poor
Learning Difficulty   = yes

Rules Fired / Conclusions:
[RULE FIRED] Academic Risk
[RULE FIRED] Attendance Improvement
[RULE FIRED] Extra Learning Support
[RULE FIRED] Assignment Support
[RULE FIRED] Faculty Counselling
[RULE FIRED] Requires Academic Attention
[NOT DERIVED] Overall Good Performance
```

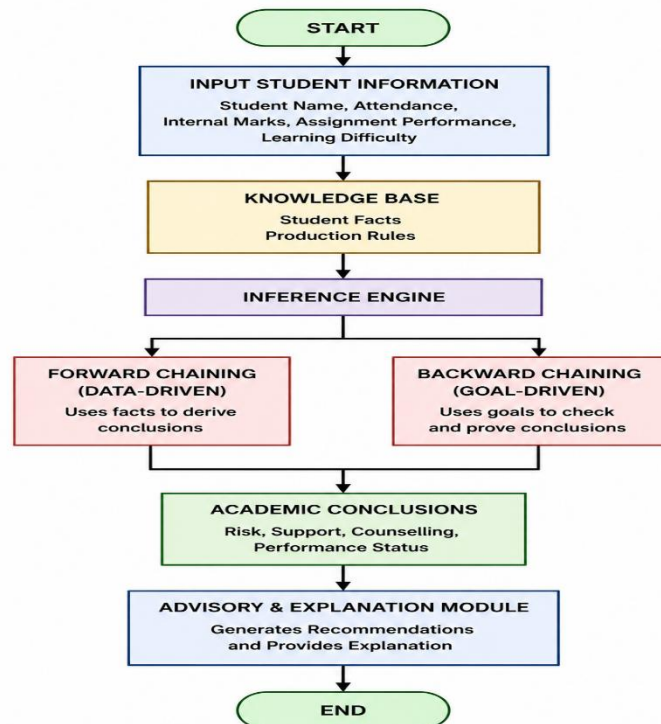
Feature	Procedural	Prolog / Declarative
Main focus	How to solve	What is true
Control flow	Explicit	Handled largely by Prolog
Knowledge representation	Procedures	Facts and rules
Reasoning	Programmer-defined sequence	Logical inference
Backtracking	Usually implemented manually	Built into Prolog
Unification	Not normally central	Core Prolog mechanism
Expert-system suitability	More implementation effort	Highly suitable

11. Prolog Implementation

- The system contains the following components:
- Knowledge Base
- Contains student facts.
- Production Rules
- Determine academic conclusions from student facts.
- Forward Chaining Module
- Starts with initial facts and derives applicable conclusions.

- Backward Chaining Module
- Checks predefined goals using goal-driven reasoning.
- Advisory Module
- Combines reasoning results and displays final recommendations.

12. System Architecture



```

=====
                        FORWARD CHAINING
=====
Initial Facts:
Attendance           = low
Internal Marks       = low
Assignment Performance = poor
Learning Difficulty  = yes

Rules Fired / Conclusions:
[RULE FIRED] Academic Risk
[RULE FIRED] Attendance Improvement
[RULE FIRED] Extra Learning Support
[RULE FIRED] Assignment Support
[RULE FIRED] Faculty Counselling
[RULE FIRED] Requires Academic Attention
[NOT DERIVED] Overall Good Performance
  
```

13. Forward Chaining Algorithm

- Accept the student's facts.
- Display the initial facts.
- Check each production rule.
- Determine whether the rule conditions are satisfied.
- Fire the applicable rule.
- Add the derived conclusion.
- Continue until all applicable rules have been processed.

14. Backward Chaining Algorithm

- Select an advisory goal.
- Search for a rule that can establish the goal.
- Check the conditions of that rule.
- Match the rule conditions with known facts.
- If the conditions are satisfied, the goal becomes TRUE.
- Otherwise, the goal becomes FALSE.
- Repeat for the required goals.
- Display the reasoning results.

15. Testing and Experimental Results

The system is tested using five student scenarios.

Updated Student Testing Table

Student	Main Academic Situation	Result
Aditya	Low attendance, low marks, poor assignments, learning difficulty	Academic risk and multiple support recommendations
Ananya	High attendance, high marks, good assignments, no learning difficulty	Overall good performance
Vikram	Low attendance, high marks, poor assignments	Academic risk and attendance/assignment support
Kavya	High attendance, low marks, good assignments, learning difficulty	Extra learning support
Rohan	Low attendance, low marks, poor assignments, learning difficulty	Academic risk and multiple recommendations

16. Main Test Case — Rohan

- Student Fact
- student(rohan, low, low, poor, yes).
- Query
- ?- advisor(rohan).
- Student Information
- Student: rohan
- Attendance: low
- Internal Marks: low
- Assignment Performance: poor
- Learning Difficulty: yes
- Derived Conclusions
- Academic Risk
- Improve Attendance
- Reasoning

Low Attendance + Low Internal Marks



Academic Risk



Faculty Counselling

Low Attendance



Improve Attendance

Learning Difficulty = Yes



Extra Learning Support

Poor Assignment Performance



Assignment Support

17. Results and Analysis

- The experimental results show that the expert system successfully applies production rules to different student scenarios.
- For Rohan, low attendance and low internal marks lead to the academic-risk conclusion.
- Low attendance produces an attendance-improvement recommendation. Learning difficulty produces extra learning support.
- Poor assignment performance produces assignment support. Academic risk subsequently produces a faculty-counselling recommendation.
- Priya represents a positive academic situation because she has high attendance, high internal marks, good assignments, and no learning difficulty.

18. Limitations

- The knowledge base contains a limited number of student attributes.
- Academic categories are represented using symbolic values such as high, low, good, and poor.
- Recommendations depend on predefined rules.
- The system does not automatically learn new rules.
- Numerical academic scores are not directly used.
- Recommendation quality depends on the correctness and completeness of the knowledge base.
- The current system is a prototype and should support, rather than replace, human academic advising.

19. Future Enhancements

- Add study hours.
- Add previous examination performance.
- Include numerical marks and attendance percentages.
- Add more academic rules.
- Add confidence levels to recommendations.
- Develop a graphical or web-based interface.
- Store student information in a database.
- Add detailed explanation facilities.
- Allow academic advisors to update the knowledge base.
- Integrate historical academic data.

20. Conclusion

- The Student Academic Advisor Expert System demonstrates how Artificial Intelligence and knowledge-based reasoning can be applied to academic advising.
- The system represents student information using Prolog facts and uses production rules to derive academic conclusions.
- Forward chaining demonstrates data-driven reasoning, while backward chaining demonstrates goal-driven reasoning.
- The project also demonstrates important Prolog concepts such as variables, unification, backtracking, and logical inference.
- Testing with Rahul, Priya, Arun, Meena, and Rohan demonstrates that the system produces different recommendations according to their academic conditions.

- For Rohan, the system identifies academic risk and recommends improvement in attendance, additional learning support, assignment support, and faculty counselling.
- Overall, the project demonstrates a rule-based expert system capable of providing understandable academic recommendations through logical reasoning.

```

=====
                ACADEMIC RECOMMENDATIONS
=====
[1] Improve Attendance
[2] Provide Extra Learning Support
[3] Improve Assignment Performance
[4] Faculty Counselling Recommended
[5] Student Requires Academic Attention
=====

=====
                EXPLANATION
=====
Academic Risk: Low attendance and low internal marks.
Attendance Support: Attendance level is low.
Learning Support: Student has learning difficulty.
Assignment Support: Assignment performance is poor.
Counselling: Academic risk requires faculty counselling.
=====
*****
                END OF REPORT
*****

```

21. References

- SWI-Prolog Documentation, Prolog language and predicates.
- Sterling, L. and Shapiro, E., *The Art of Prolog: Advanced Programming Techniques*.
- Bratko, I., *Prolog Programming for Artificial Intelligence*.
- Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*.
- Course materials on Artificial Intelligence and Expert Systems.

23. GitHub Repository

URL: <https://github.com/Sathwik052007/Sathwik---MLA0103.git>

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.

3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.

7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.
9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				